

Contents

Introduction	4
1. Service Engineering Process	5
1.1 Service Candidate Identification	5
Patient Service (Adham)	5
Doctor Service (Andrew)	5
Appointment Service (Maged)	5
Service Classification Table	6
1.2 Service Requirements Specification	6
Patient Service Requirements (Adham - A2)	6
Doctor Service Requirements (Andrew - B2)	7
Appointment Service Requirements (Maged - M0)	7
1.3 Service Interface Design	8
REST API Endpoints Summary	8
2. API Design	9
2.1 Resources and Endpoints	9
Resource: Patient (/api/patients)	9
Resource: Doctor (/api/doctors)	9
Resource: Appointment (/api/appointments)	9
Resource: Composite booking (/api/composite/book-appointment)	10
2.2 HTTP Methods Mapping	10
Status Codes	10
3. Service Composition	11
3.1 Composite Service Description	11
Service Name: Book Appointment	11
Description	11
Orchestrated Services	11
3.2 Workflow	11
Text sequence (client → system)	11
Steps	12

3.3 Implementation Notes	12
4. Testing	12
4.1 GET API Tests (Adham - A3)	13
Test Case 1: GET /api/patients	13
Test Case 2: GET /api/patients/{id}	14
4.2 POST API Tests (Andrew - B3)	15
POST /api/doctors	15
POST /api/appointments	16
4.3 PUT API Tests (Yassin - Y2)	17
PUT /api/patients/{id}	17
PUT /api/appointments/{id}	18
4.4 DELETE API Tests (Maged - M1)	19
DELETE /api/appointments/{id}	19
DELETE /api/patients/{id}	20
4.5 Composite Service Test (Andrew - B4)	21
POST /api/composite/book-appointment	21
4.6 Error Handling Tests (Yassin - Y3)	22
GET unknown patient	22
POST invalid patient	23
POST duplicate email	24
5. API Documentation (OpenAPI)	24
5.1 OpenAPI Specification Overview	24
5.2 YAML Snippet (representative excerpt)	25
5.3 How to Use	26
6. Implementation	26
6.1 Technology Stack	26
6.2 Project Structure	26
6.3 Layered Architecture	27
7. Assumptions	27
8. Work Division	27
Appendix A: Jira Screenshots	28

Appendix B: Complete OpenAPI YAML.....	28
API Documentation and Swagger UI.....	38
Swagger UI Implementation	38
API Endpoints Overview	38
OpenAPI Specification.....	39
Info Section	39
Schemas Defined	39
Screenshots Placeholders	40
Documentation Features	42
Web GUI Implementation	42
Access and Features	42
GUI Capabilities.....	42
Patient Management.....	42
Doctor Management.....	42
Appointment System	43
Composite Booking.....	43
Technical Integration.....	43
GUI	44
SECTION 2: Database Configuration	46
Database Configuration	46
Database Setup	46
Database Schema.....	46
Pre-loaded Data	47
Sample Patients	47
Sample Doctors	47
Data Management Features	47
Console Access.....	47
SECTION 3: Error Handling Details.....	48
Error Handling Strategy.....	48
HTTP Status Codes	48
400 Bad Request	48

404 Not Found	48
409 Conflict	48
200 Success	49
201 Created	49
Error Response Format	49
Validation Examples	49
Patient Validation	49
Appointment Validation	50
Error Prevention.....	50
Appendix C: Evidence Folder Contents	50

Introduction

This document is the technical report for **Deliverable 5: Service-Oriented Software Engineering** for the **Hospital Management System (HMS)**. The team applied the service engineering lifecycle (**Identify → Define → Design → Implement → Deploy → Validate**) to

produce reusable **RESTful** services for patients, doctors, and appointments, plus one **composite** workflow that books an appointment by orchestrating multiple APIs.

The implementation is a **Spring Boot 3.2.5** application (Java 17) using **Spring Web**, **Spring Data JPA**, **H2** (in-memory), and **Jakarta Bean Validation**. APIs were exercised with **Postman**; machine-readable API documentation is provided as **OpenAPI 3.0.3 YAML** (`Implementation/openapi.yaml`) and summarized here. Evidence text artifacts in the `Evidence/` folder support traceability from requirements to design.

1. Service Engineering Process

1.1 Service Candidate Identification

Patient Service (Adham)

The Patient Service manages the lifecycle of patient records (registration, retrieval, update, delete). It is a strong service candidate because:

- **(a) reusability** — patient data is consumed by appointment scheduling and by the composite “book appointment” flow.
- **(b) independence** — patient CRUD does not require doctor or appointment business rules.
- **(c) boundary** — a single cohesive capability (patient registry).
- **(d) external usage** — billing, lab, or pharmacy systems could reuse the same API.

Doctor Service (Andrew)

The Doctor Service maintains authoritative doctor profiles (identity, contact, specialty, availability). It is justified by:

- **(a) reusability** — consumed by scheduling, reporting, and composite booking.
- **(b) independence** — doctor CRUD is orthogonal to appointment persistence.
- **(c) boundary** — clear domain slice (clinical staff directory).
- **(d) reuse** — portals and rostering can integrate without duplicating doctor persistence.

Appointment Service (Maged)

The Appointment Service stores scheduled encounters linking `patientId`, `doctorId`, `date`, `time`, `reason`, and `status`. It is justified by:

- **(a) reusability** — shared by front desk, doctor views, and composite booking.
- **(b) independence** — exposes its own REST contract and persistence.
- **(c) boundary** — scheduling aggregate (who/when/why).

- **(d) coordination** — coordinates actors across the hospital even when patient/doctor masters live in sibling services, so it is classified as a **Coordination Service**.

Service Classification Table

Service	Type	Justification
Patient Service	Business	Core hospital master data; directly supports clinical and administrative processes; high reuse across workflows.
Doctor Service	Business	Core master data for providers; reused by scheduling and composite flows.
Appointment Service	Coordination	Links patient and doctor in time; bridges multiple business actors while keeping masters in Patient/Doctor services.

1.2 Service Requirements Specification

Patient Service Requirements (Adham - A2)

Functional Requirements

ID	Requirement	Priority
FR-P1	The system shall allow retrieving a list of all registered patients (<code>GET /api/patients</code>).	Must
FR-P2	The system shall allow retrieving a single patient by ID (<code>GET /api/patients/{id}</code>).	Must
FR-P3	The system shall allow registering, updating, and deleting patients (<code>POST</code> , <code>PUT</code> , <code>DELETE</code>).	Must

Non-Functional Requirements

ID	Requirement	Target
NFR-P1	Performance — list patients	≤ 500 ms for up to ~1,000 rows (design target under classroom hardware).
NFR-P2	Data integrity — email uniqueness	Duplicate email returns 409 Conflict .
NFR-P3	Reliability — HTTP semantics	Consistent use of 200 / 201 / 204 / 400 / 404 / 409 per REST practice.
NFR-P4	Maintainability	Layered architecture: Controller $\rightarrow$ Service $\rightarrow$ Repository; business rules not in controllers.
NFR-P5	Usability of errors	JSON errors include <code>timestamp</code> , <code>status</code> , <code>error</code> , <code>message</code> (and field <code>details</code> for validation failures).

Inputs, Outputs, Constraints

- **Input:** JSON bodies for create/update (`PatientRequest`: `name`, `email`, `phone`; optional `birthDate`, `gender`). Path parameter `id` for single-resource operations.

- **Output:** `PatientDTO` JSON for reads/creates/updates; **204 No Content** on successful delete.
- **Constraints:** Unique email; phone pattern validation; IDs are system-generated; H2 in-memory database for this deliverable.

Doctor Service Requirements (Andrew - B2)

Functional Requirements

ID	Requirement	Priority
FR-D1	List and get doctors (GET <code>/api/doctors</code> , GET <code>/api/doctors/{id}</code>).	Must
FR-D2	Create, update, delete doctors (POST, PUT, DELETE).	Must

Non-Functional Requirements

ID	Requirement	Target
NFR-D1 Performance	Responsive list/get for typical roster sizes.	
NFR-D2 Data integrity	Unique doctor email; conflicts return 409 .	
NFR-D3 Reliability	Consistent HTTP status codes including 400 validation.	
NFR-D4 Maintainability	<code>DoctorController</code> → <code>DoctorService</code> → <code>DoctorRepository</code> .	
NFR-D5 Error clarity	JSON error bodies suitable for API clients.	

Inputs, Outputs, Constraints

- **Input:** `DoctorRequest` (name, email, phone, specialty; optional `available` defaulting to true).
- **Output:** `DoctorDTO`; **204** on delete.
- **Constraints:** Unique email; H2 in-memory store.

Appointment Service Requirements (Maged - M0)

Functional Requirements

ID	Requirement	Priority
FR-A1	List and get appointments (GET <code>/api/appointments</code> , GET <code>/api/appointments/{id}</code>).	Must
FR-A2	Create, update, delete appointments (POST, PUT, DELETE).	Must

Non-Functional Requirements

ID	Requirement	Target
NFR-A1 Performance	Responsive under normal demo/Postman load.	
NFR-A2 Data integrity	Required fields enforced; 400 with field errors on validation failure.	
NFR-A3 Reliability	200 / 201 / 204 / 404 used predictably.	

ID	Requirement	Target
NFR-A4 Maintainability	Controller → Service → Repository with DTO mapping.	
NFR-A5 Error consistency	Same global error style as other services.	

Inputs, Outputs, Constraints

- **Input:** AppointmentRequest (patientId, doctorId, date, timeSlot, optional reason).
- **Output:** AppointmentDTO including server-managed status (e.g. **CONFIRMED** on direct create in implementation).
- **Constraints:** MVP does not enforce DB-level FK to patient/doctor tables; composite flow validates existence before insert. Duplicate slot constraint is a recommended production rule, not enforced in MVP code.

1.3 Service Interface Design

REST API Endpoints Summary

Service	Endpoint	Method	Description
Patient	/api/patients	GET	Get all patients
Patient	/api/patients/{id}	GET	Get patient by ID
Patient	/api/patients	POST	Create patient
Patient	/api/patients/{id}	PUT	Update patient
Patient	/api/patients/{id}	DELETE	Delete patient
Doctor	/api/doctors	GET	Get all doctors
Doctor	/api/doctors/{id}	GET	Get doctor by ID
Doctor	/api/doctors	POST	Create doctor
Doctor	/api/doctors/{id}	PUT	Update doctor
Doctor	/api/doctors/{id}	DELETE	Delete doctor
Appointment	/api/appointments	GET	Get all appointments
Appointment	/api/appointments/{id}	GET	Get appointment by ID
Appointment	/api/appointments	POST	Create appointment
Appointment	/api/appointments/{id}	PUT	Update appointment
Appointment	/api/appointments/{id}	DELETE	Delete appointment
Composite	/api/composite/book-appointment	POST	Book appointment (composite)

2. API Design

2.1 Resources and Endpoints

Resource: Patient (`/api/patients`)

Represents a person receiving care. Fields in `PatientDTO` / entity:

- `id`
- `name`
- `email`
- `phone`
- `optional birthDate`
- `optional gender`

The resource collection is `/api/patients`; an individual resource is `/api/patients/{id}`.

Resource: Doctor (`/api/doctors`)

Represents a provider. Fields:

- `id`
- `name`
- `email`
- `phone`
- `specialty`
- `available (boolean)`

Collection and item URIs follow the same pattern as patients.

Resource: Appointment (`/api/appointments`)

Represents a scheduled encounter. Fields:

- `id`
- `patientId`
- `doctorId`
- `date (string YYYY-MM-DD)`
- `timeSlot (string HH:mm)`
- `optional reason`
- `status`

Collection `/api/appointments`; item `/api/appointments/{id}`.

Resource: Composite booking (`/api/composite/book-appointment`)

Not a persisted “resource collection” in the REST sense; it is a **process endpoint** that triggers a workflow.

Request body:

- `patientId`
- `doctorId`
- `date`
- `timeSlot`
- optional `reason`

Response: composite DTO including appointment id plus patient/doctor summary fields (see §3).

2.2 HTTP Methods Mapping

We follow **REST** conventions:

- **nouns** identify resources
- **HTTP verbs** express intent

Method	Role	HMS usage
GET	Safe retrieval; no server state change	List (<code>.../patients</code>) or read one (<code>.../patients/{id}</code>).
POST	Create subordinate resource / non-idempotent actions	<code>POST /api/patients</code> creates a new patient; composite uses <code>POST</code> for process execution.
PUT	Replace/update resource identified by URL	<code>PUT .../patients/{id}</code> and <code>PUT .../appointments/{id}</code> .
DELETE	Remove resource	<code>DELETE .../patients/{id}</code> returns 204 with empty body on success.

Status Codes

- **201 Created** after successful `POST` creates
- **200 OK** for successful `GET` / `PUT`
- **204 No Content** for successful `DELETE`
- **400** for validation
- **404** when an entity is missing

- **409** for duplicate email (patient/doctor)

3. Service Composition

3.1 Composite Service Description

Service Name: Book Appointment

Description

The composite **Book Appointment** operation lets a client submit one JSON payload to complete a booking workflow.

The server **orchestrates** validation against **Patient** and **Doctor** data, then **creates** an **Appointment**.

This matches the course requirement: a composite service that **orchestrates two or more APIs** and represents a **business process** (schedule a visit).

Orchestrated Services

1. **Patient** — verify `patientId` exists (read path via repository in the monolith; logically `GET /api/patients/{id}`).
2. **Doctor** — verify `doctorId` exists (logical `GET /api/doctors/{id}`).
3. **Appointment** — persist booking (logical `POST /api/appointments`).

3.2 Workflow

Text sequence (client → system)

Client	CompositeController	Data stores
-- POST /api/composite/ ----->		
book-appointment		
	-- load patient ----->	
	<- 404 fail / OK -----	
	-- load doctor ----->	
	<- 404 fail / OK -----	
	-- save appointment ->	
	<- saved row -----	
<- 201 BookingResponse -----		

Steps

1. **Receive booking request** — JSON: `patientId`, `doctorId`, `date`, `timeSlot`, optional `reason`. Validation failures → **400** with field details.
2. **Validate patient** — If missing → **404** (Patient not found: {id} style message in implementation).
3. **Validate doctor** — If missing → **404** (Doctor not found: {id}).
4. **Create appointment** — Persist row with `status` set to **CONFIRMED** in the composite path.
5. **Return composite response** — **201 Created** with body including:
 - o `appointmentId`
 - o `patientId`
 - o `patientName`
 - o `doctorId`
 - o `doctorName`
 - o `doctorSpecialty`
 - o `date`
 - o `timeSlot`
 - o `reason`
 - o `status`

(flattened DTO in code; design doc A4 showed a nested view—both describe the same workflow).

3.3 Implementation Notes

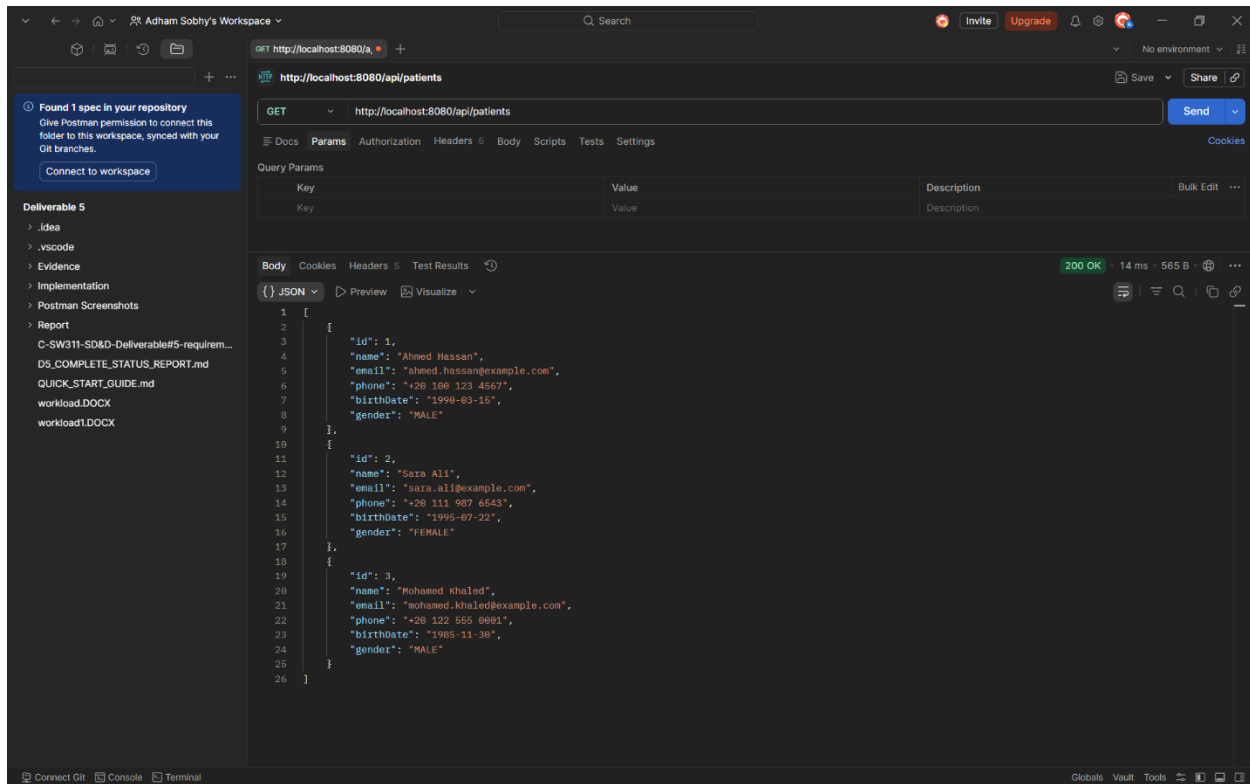
- **Orchestration vs choreography:** Central controller coordinates sequential steps; easier to trace in Postman than event-driven choreography.
- **Fail-fast:** No appointment insert if patient or doctor validation fails.
- **No distributed transaction:** Steps 2–3 are reads; if persistence fails, client receives error without compensating transaction (acceptable scope for D5).
- **Monolith mapping:** For simplicity, the composite uses repositories in-process; logically it matches the multi-service sequence in evidence **Y0 / A4**.

4. Testing

All primary tests were executed with **Postman** against `http://localhost:8080` while the Spring Boot application was running.

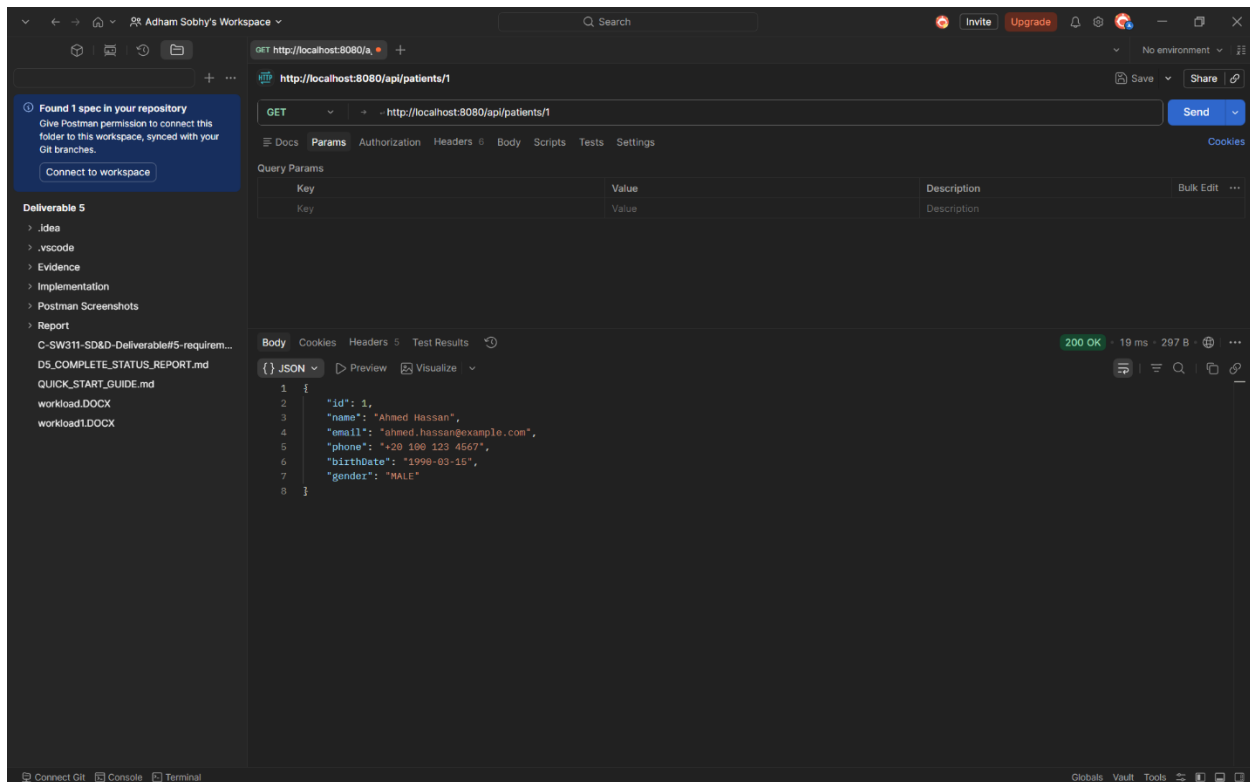
Screenshots are stored in `Postman Screenshots/` and embedded below.

4.1 GET API Tests (Adham - A3)



Test Case 1: GET /api/patients

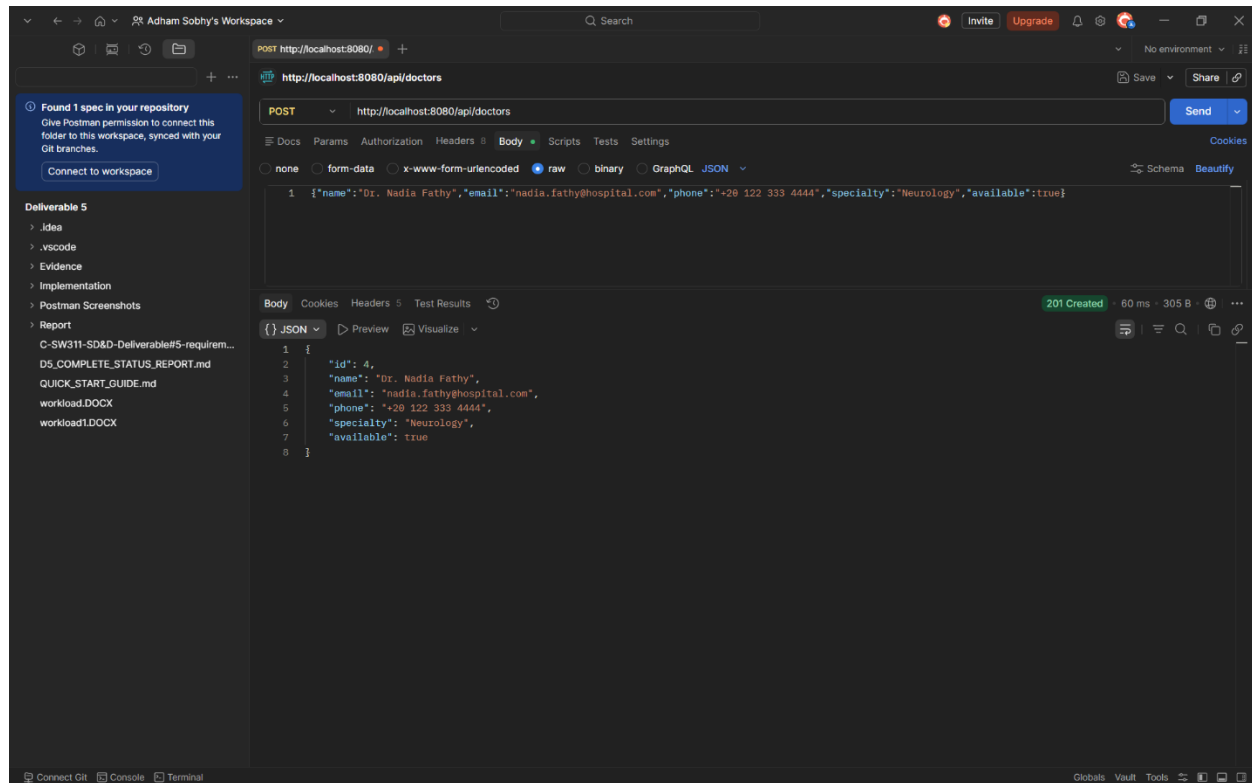
- **Expected:** 200 OK, JSON array of patients (seeded data present after startup).
- **Actual:** 200 OK; array includes patients with ids 1..3 and expected emails from DataInitializer.



Test Case 2: GET /api/patients/{id}

- **Expected:** 200 OK, single PatientDTO for existing id.
- **Actual:** 200 OK for GET /api/patients/1 with full patient fields.

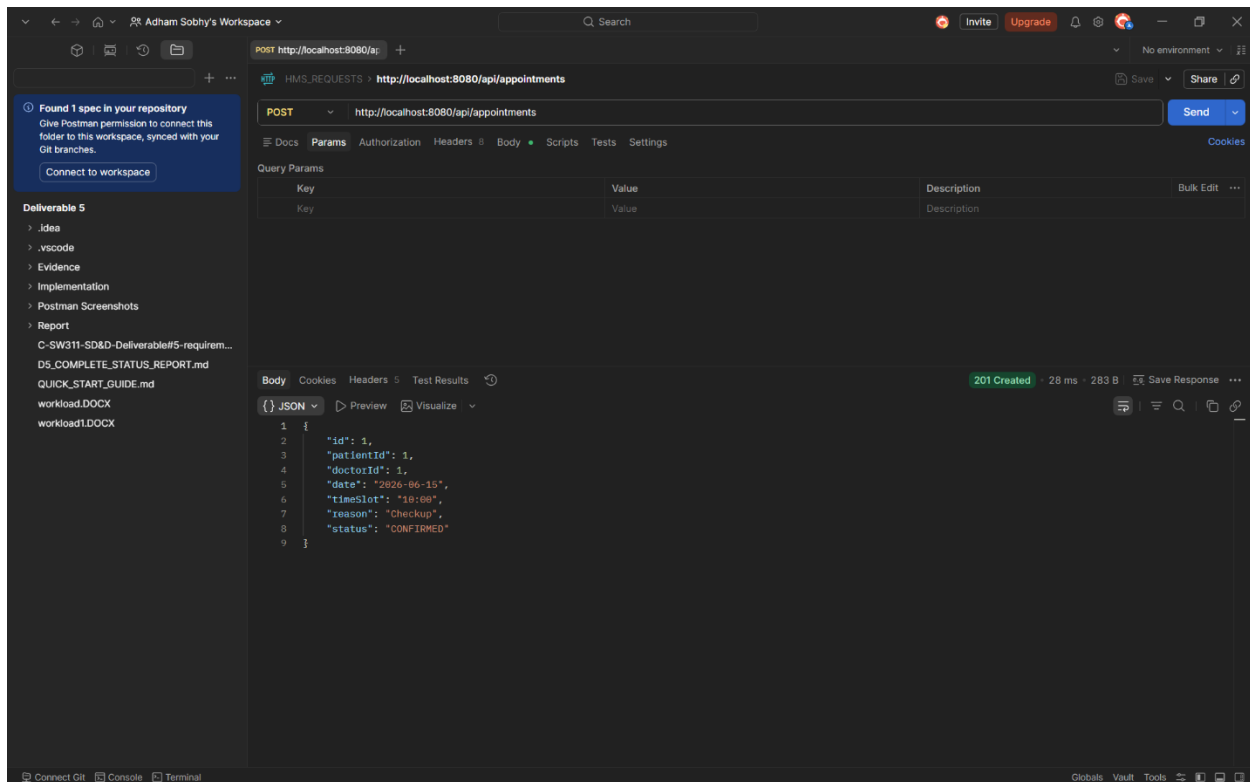
4.2 POST API Tests (Andrew - B3)



POST /api/doctors

Creates a doctor with JSON body (`DoctorRequest`).

Expected 201 **created** with returned `DoctorDTO` when email is unique.

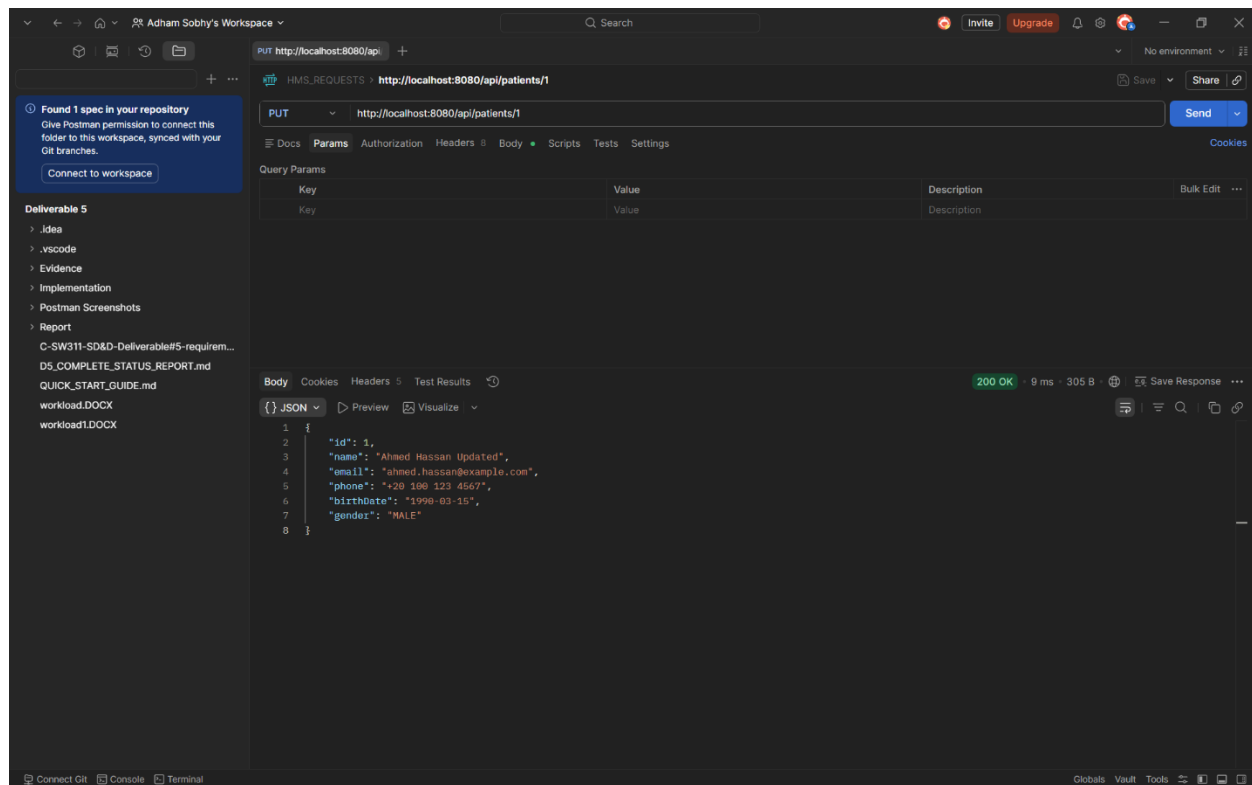


POST /api/appointments

Creates appointment with `AppointmentRequest`.

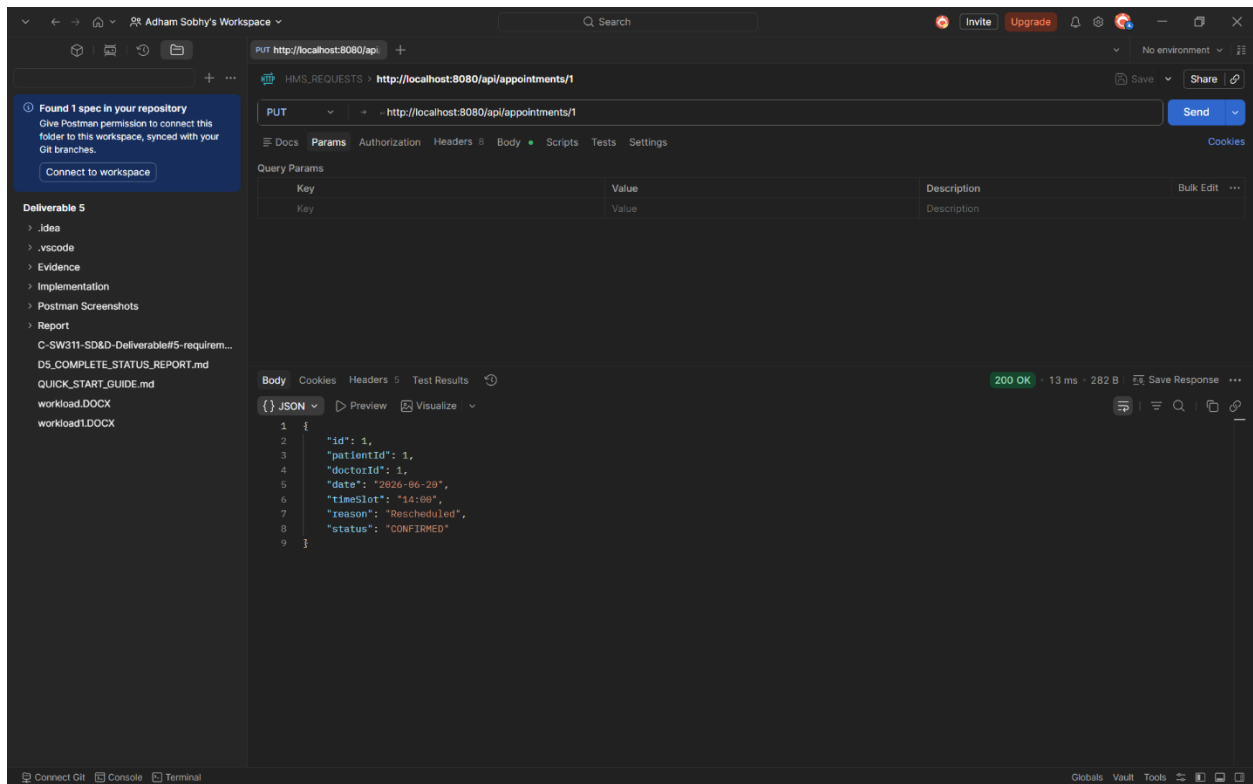
Expected **201 created** with persisted `AppointmentDTO`.

4.3 PUT API Tests (Yassin - Y2)



PUT /api/patients/{id}

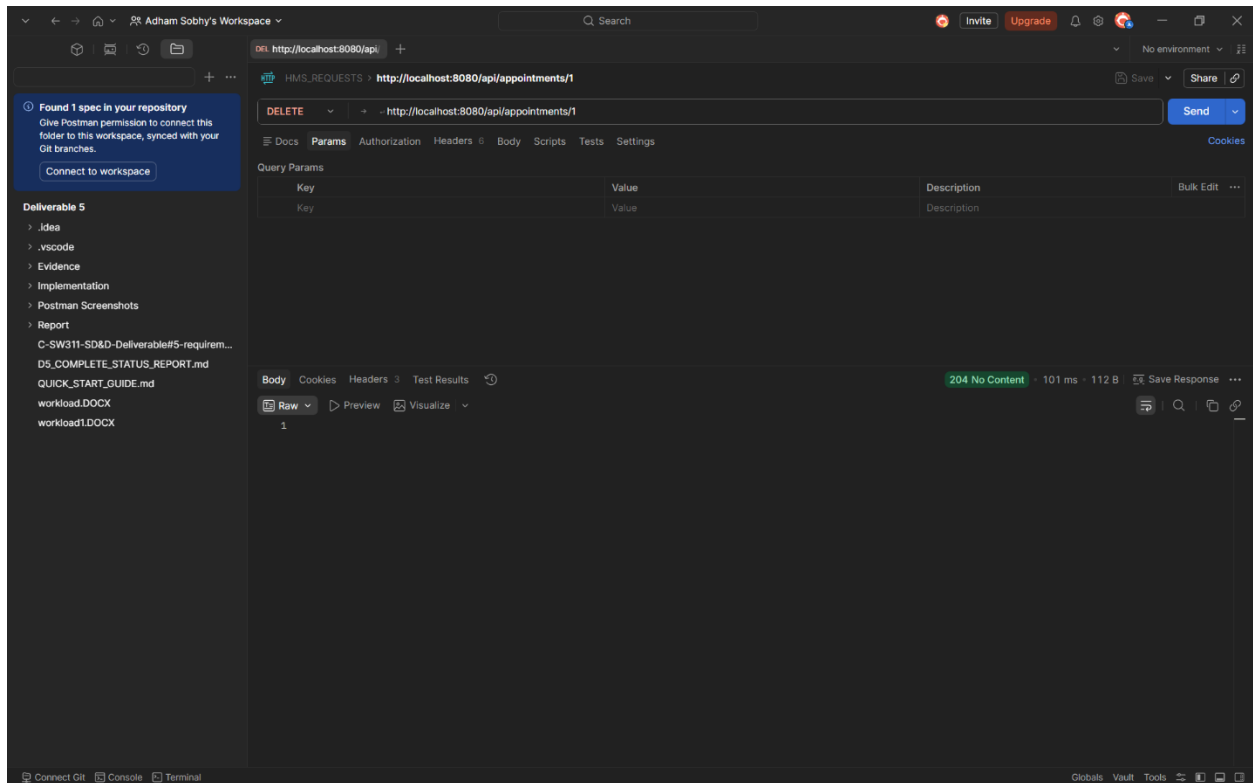
Updates patient fields; expected `200 OK` with updated DTO when validation passes and email rules satisfied.



PUT /api/appointments/{id}

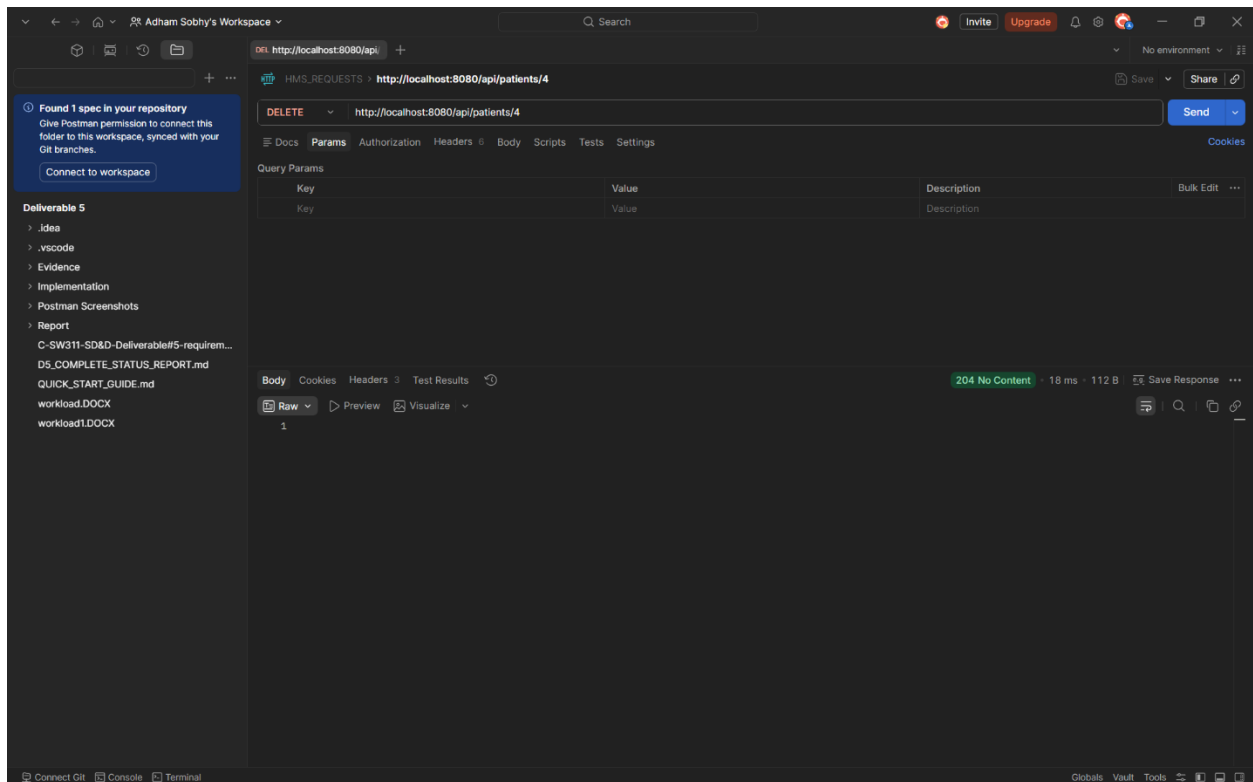
Reschedules / updates appointment; expected 200 ok.

4.4 DELETE API Tests (Maged - M1)



DELETE /api/appointments/{id}

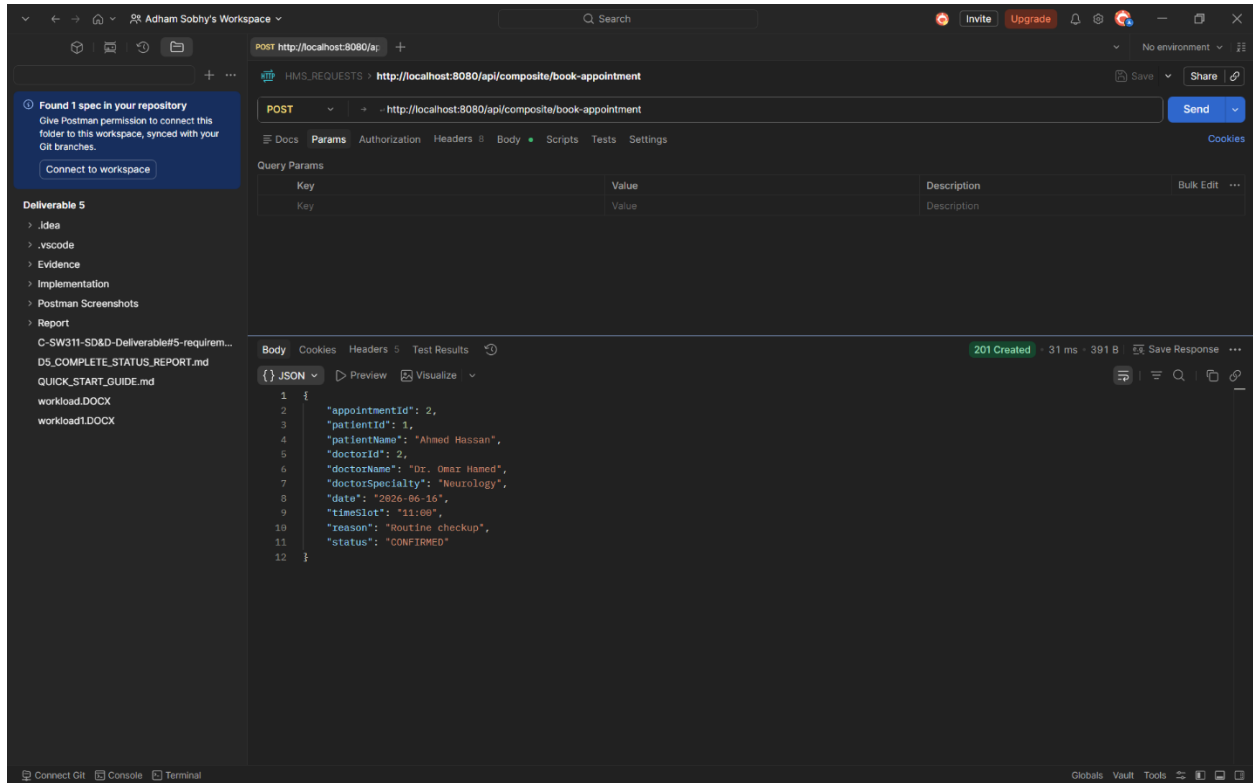
Expected **204 No Content** on success.



DELETE /api/patients/{id}

Expected **204 No Content** after creating a disposable patient if needed (ids 1–3 are seeded; tests used a newly created id where applicable).

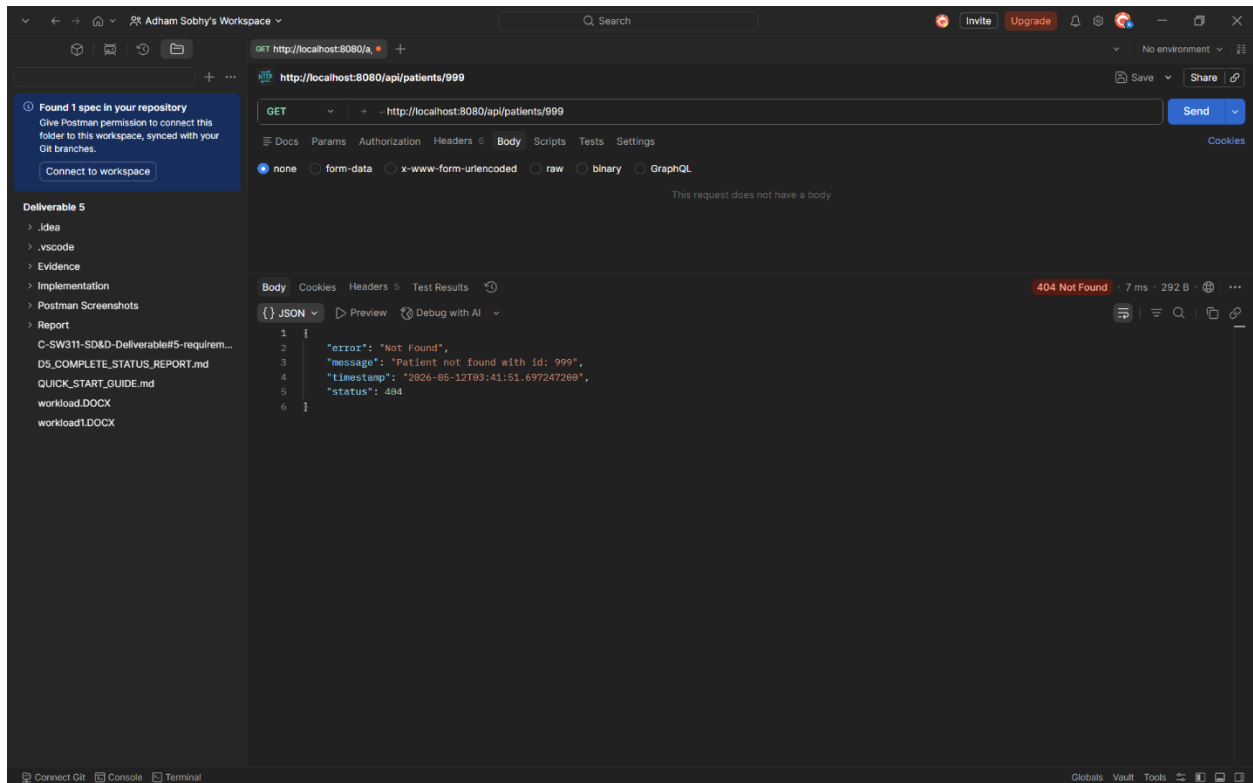
4.5 Composite Service Test (Andrew - B4)



POST /api/composite/book-appointment

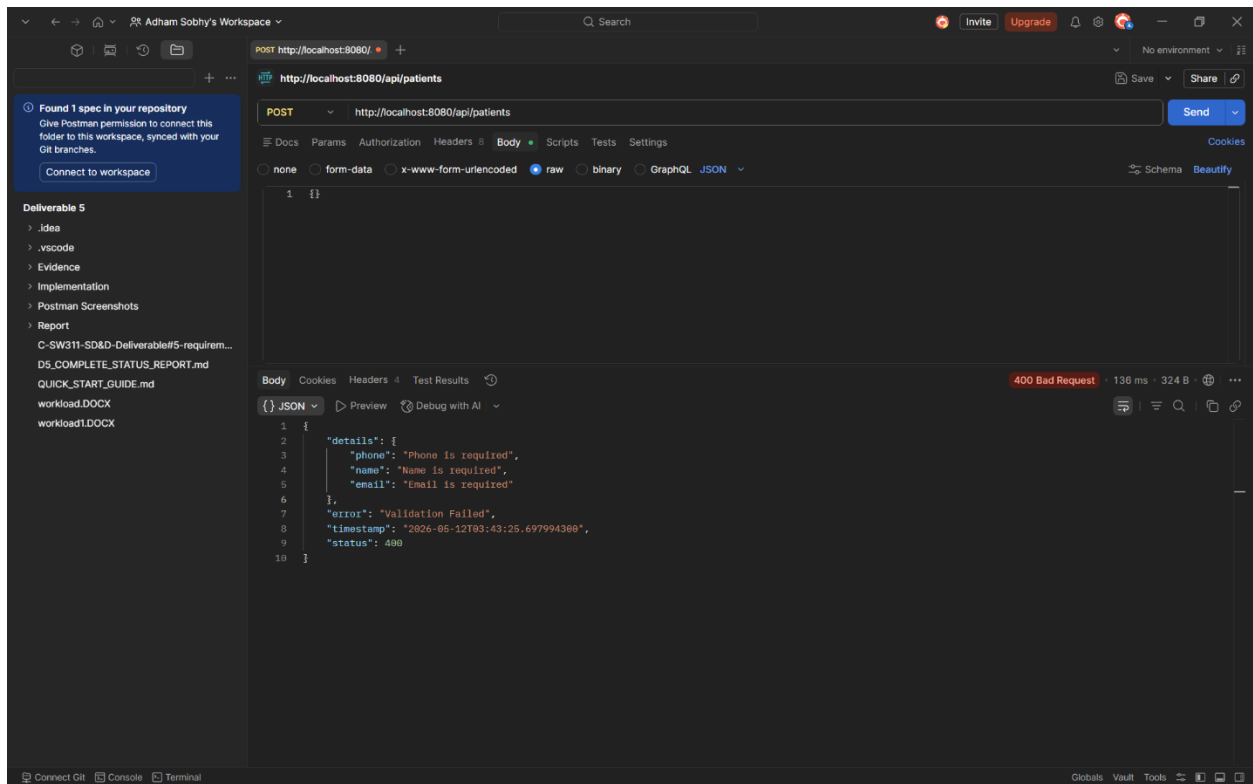
With valid `patientId` / `doctorId`, expected **201 created** and composite JSON including `appointmentId` and names/specialty fields.

4.6 Error Handling Tests (Yassin - Y3)



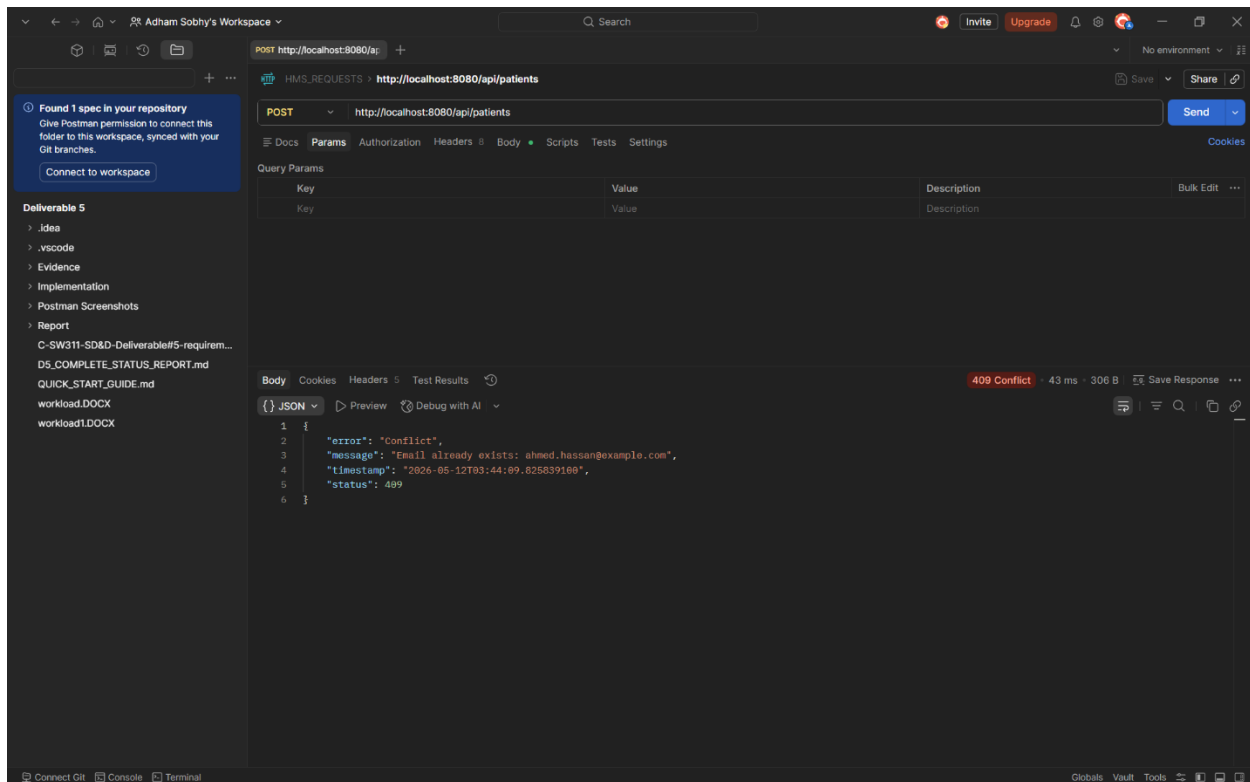
GET unknown patient

Example: `/api/patients/999` → **404 Not Found** with JSON message.



POST invalid patient

Empty or invalid body → **400 Bad Request** with `details` map for fields.



POST duplicate email

Reuse existing seeded email → **409 Conflict**.

5. API Documentation (OpenAPI)

5.1 OpenAPI Specification Overview

The canonical specification is `Implementation/openapi.yaml` (OpenAPI 3.0.3).

It declares:

- **info** — title, description, version, contact.
- **servers** — local base URL `http://localhost:8080`.
- **paths** — all REST paths for patients, doctors, appointments, and composite booking, including:
 - **parameters**
 - **requestBody** schemas
 - **responses** with status codes (**200, 201, 204, 400, 404, 409** as applicable)
- **components.schemas** — reusable DTOs:
 - `PatientDTO`

- o PatientRequest
- o DoctorDTO
- o DoctorRequest
- o AppointmentDTO
- o AppointmentRequest
- o BookingRequest
- o BookingResponse

5.2 YAML Snippet (representative excerpt)

```

openapi: 3.0.3
info:
  title: Hospital Management System API
  version: 1.0.0

servers:
  - url: http://localhost:8080

paths:
  /api/patients:
    get:
      tags: [Patient Service]
      responses:
        '200':
          description: List of patients

    post:
      tags: [Patient Service]
      requestBody:
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/PatientRequest'

      responses:
        '201':
          description: Patient created

        '409':
          description: Duplicate email

  /api/composite/book-appointment:
    post:
      tags: [Composite Service]

      requestBody:
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/BookingRequest'

      responses:
        '201':
          description: Booking confirmed

        '404':

```

```
        description: Patient or doctor not found

components:
  schemas:
    PatientRequest:
      type: object
      required: [name, email, phone]
```

(Full file reproduced in **Appendix B.**)

5.3 How to Use

1. **Postman** — Import nothing required; create requests for GET/POST/PUT/DELETE on `/api/...` with JSON bodies matching `PatientRequest`, `DoctorRequest`, `AppointmentRequest`, or composite `BookingRequest`.
 2. **Swagger UI (optional bonus)** — With the app running, open `http://localhost:8080/swagger-ui/index.html` (SpringDoc) to use **Try it out** against the live server.
 3. **H2 Console** — `http://localhost:8080/h2-console` with JDBC URL `jdbc:h2:mem:hmsdb`, user `sa`, empty password (for debugging only).
-

6. Implementation

6.1 Technology Stack

Component	Technology	Version
Framework	Spring Boot	3.2.5
Language	Java	17
Database	H2 (in-memory)	2.x (via Spring Boot BOM)
Build Tool	Maven	3.9.x (wrapper available)
Validation	Jakarta Bean Validation	3.x
Utilities	Lombok	Managed by Spring Boot
API docs (bonus)	SpringDoc OpenAPI	2.6.0

6.2 Project Structure

```
Implementation/src/main/java/com/mycompany/hms/
├── controller/      # REST controllers (Patient, Doctor, Appointment,
Composite)
├── service/         # Interfaces + implementations
├── repository/      # Spring Data JPA repositories
├── model/           # JPA entities
```

```

├─ dto/           # Request/response DTOs
├─ exception/     # Custom exceptions + GlobalExceptionHandler
├─ config/        # DataInitializer (+ optional OpenAPI config)

```

6.3 Layered Architecture

Requests hit a `@RestController`, which delegates to a `@Service` implementation containing validation and domain rules (e.g. duplicate email checks). Services use `JpaRepository` interfaces for persistence. Entities map to the H2 schema created by JPA (`ddl-auto=create-drop` for this course demo). Global exception handling translates domain exceptions and validation failures into consistent JSON HTTP responses.

7. Assumptions

1. **H2 in-memory database** is acceptable for D5; data resets on application restart (`create-drop` / in-memory scope).
 2. **No authentication/authorization** — APIs are open on `localhost` for demonstration.
 3. **IDs** are `Long`, generated by the database; clients never choose ids on create.
 4. **Dates and time slots** are carried as **strings** (`YYYY-MM-DD, HH:mm`) for simplicity and JSON portability.
 5. **Appointment** records reference `patientId/doctorId` without database-level foreign keys to other tables in this MVP; integrity for composite booking is ensured by orchestration reads before insert.
 6. **No duplicate-slot enforcement** in MVP code (recommended for production).
 7. **English field names and JSON** for interoperability; locale-specific formatting is out of scope.
 8. **Team evidence** (`Evidence/*.txt`) aligns with implemented endpoints; minor differences between idealized nested composite response (A4) and flattened runtime DTO are documented in §3.
-

8. Work Division

Member	Tasks	Code / artifacts	Report ownership
Adham	A0, A2, A3, A4, A5	Patient APIs, composite design, assembly	Introduction, patient service engineering, GET tests, coordination
Andrew	B0, B1, B2, B3, B4, B5	Doctor APIs, composite controller, Jira	Doctor requirements, POST tests, composite test, Jira appendix
Yassin	Y0, Y1, Y2, Y3	Data seeding, PUT APIs, exception handling	PUT tests, error tests, infra notes

Member	Tasks	Code / artifacts	Report ownership
Maged	M0, M1, M2	Appointment design, DELETE, OpenAPI YAML	Appointment requirements, DELETE tests, OpenAPI sections

(Replace [Lastname] for Andrew, Yassin, and Maged before PDF export.)

Appendix A: Jira Screenshots

If the image above does not render, add your exported screenshot as Evidence/B0_Jira_Screenshots/board.png or update the path to match your filenames.

Appendix B: Complete OpenAPI YAML

```
openapi: 3.0.3

info:
  title: Hospital Management System API
  description: |
    RESTful API for the Hospital Management System – Deliverable 5.
    Services: Patient, Doctor, Appointment, and Composite Booking.
  version: 1.0.0

  contact:
    name: SW311 Team

servers:
  - url: http://localhost:8080
    description: Local Spring Boot server

paths:
  # =====
  # Patient Service
  # =====

  /api/patients:
    get:
      tags: [Patient Service]
      summary: Get all patients
      operationId: getAllPatients

      responses:
        '200':
          description: List of patients

          content:
```

```

        application/json:
          schema:
            type: array

        example:
          - id: 1
            name: "Ahmed Hassan"
            email: "ahmed.hassan@example.com"
            phone: "+20 100 123 4567"
            birthDate: "1990-03-15"
            gender: "MALE"

    post:
      tags: [Patient Service]
      summary: Create a patient
      operationId: createPatient

      requestBody:
        required: true

        content:
          application/json:
            schema:
              $ref: '#/components/schemas/PatientRequest'

      responses:
        '201':
          description: Patient created

          content:
            application/json:
              schema:
                $ref: '#/components/schemas/PatientDTO'

        '400':
          description: Validation error

        '409':
          description: Duplicate email

    /api/patients/{id}:
      get:
        tags: [Patient Service]
        summary: Get patient by ID
        operationId: getPatientById

        parameters:
          - name: id
            in: path
            required: true

          schema:
            type: integer
            format: int64

        responses:
          '200':

```

```

        description: Patient found

        content:
          application/json:
            schema:
              $ref: '#/components/schemas/PatientDTO'

      '404':
        description: Patient not found

    put:
      tags: [Patient Service]
      summary: Update a patient
      operationId: updatePatient

      parameters:
        - name: id
          in: path
          required: true

          schema:
            type: integer
            format: int64

      requestBody:
        required: true

        content:
          application/json:
            schema:
              $ref: '#/components/schemas/PatientRequest'

      responses:
        '200':
          description: Patient updated

        '404':
          description: Patient not found

    delete:
      tags: [Patient Service]
      summary: Delete a patient
      operationId: deletePatient

      parameters:
        - name: id
          in: path
          required: true

          schema:
            type: integer
            format: int64

      responses:
        '204':
          description: Patient deleted

```

```

        '404':
            description: Patient not found

# =====
# Doctor Service
# =====

/api/doctors:
    get:
        tags: [Doctor Service]
        summary: Get all doctors
        operationId: getAllDoctors

        responses:
            '200':
                description: List of doctors

                content:
                    application/json:
                        schema:
                            type: array

                        example:
                            - id: 1
                              name: "Dr. Fatima Zaki"
                              email: "fatima.zaki@hospital.com"
                              specialty: "Cardiology"
                              available: true

    post:
        tags: [Doctor Service]
        summary: Create a doctor
        operationId: createDoctor

        requestBody:
            required: true

            content:
                application/json:
                    schema:
                        $ref: '#/components/schemas/DoctorRequest'

        responses:
            '201':
                description: Doctor created

            '400':
                description: Validation error

/api/doctors/{id}:
    get:
        tags: [Doctor Service]
        summary: Get doctor by ID
        operationId: getDoctorById

        parameters:
            - name: id

```

```
    in: path
    required: true

    schema:
      type: integer
      format: int64

  responses:
    '200':
      description: Doctor found

    '404':
      description: Doctor not found

put:
  tags: [Doctor Service]
  summary: Update a doctor
  operationId: updateDoctor

  parameters:
    - name: id
      in: path
      required: true

    schema:
      type: integer
      format: int64

  requestBody:
    required: true

    content:
      application/json:
        schema:
          $ref: '#/components/schemas/DoctorRequest'

  responses:
    '200':
      description: Doctor updated

    '404':
      description: Doctor not found

delete:
  tags: [Doctor Service]
  summary: Delete a doctor
  operationId: deleteDoctor

  parameters:
    - name: id
      in: path
      required: true

    schema:
      type: integer
      format: int64
```



```

    responses:
      '204':
        description: Doctor deleted

      '404':
        description: Doctor not found

# =====
# Appointment Service
# =====

/api/appointments:
  get:
    tags: [Appointment Service]
    summary: Get all appointments
    operationId: getAllAppointments

    responses:
      '200':
        description: List of appointments

  post:
    tags: [Appointment Service]
    summary: Create an appointment
    operationId: createAppointment

    requestBody:
      required: true

      content:
        application/json:
          schema:
            $ref: '#/components/schemas/AppointmentRequest'

    responses:
      '201':
        description: Appointment created

      '400':
        description: Validation error

/api/appointments/{id}:
  get:
    tags: [Appointment Service]
    summary: Get appointment by ID
    operationId: getAppointmentById

    parameters:
      - name: id
        in: path
        required: true

      schema:
        type: integer
        format: int64

    responses:

```

```

    '200':
      description: Appointment found

    '404':
      description: Appointment not found

put:
  tags: [Appointment Service]
  summary: Update an appointment
  operationId: updateAppointment

  parameters:
    - name: id
      in: path
      required: true

    schema:
      type: integer
      format: int64

  requestBody:
    required: true

    content:
      application/json:
        schema:
          $ref: '#/components/schemas/AppointmentRequest'

  responses:
    '200':
      description: Appointment updated

    '404':
      description: Appointment not found

delete:
  tags: [Appointment Service]
  summary: Delete an appointment
  operationId: deleteAppointment

  parameters:
    - name: id
      in: path
      required: true

    schema:
      type: integer
      format: int64

  responses:
    '204':
      description: Appointment deleted

    '404':
      description: Appointment not found

# =====

```

```

# Composite Service
# =====

/api/composite/book-appointment:
  post:
    tags: [Composite Service]
    summary: Book Appointment (orchestrates Patient + Doctor + Appointment)
    operationId: bookAppointment

    requestBody:
      required: true

      content:
        application/json:
          schema:
            $ref: '#/components/schemas/BookingRequest'

    responses:
      '201':
        description: Booking confirmed

        content:
          application/json:
            schema:
              $ref: '#/components/schemas/BookingResponse'

      '400':
        description: Validation error

      '404':
        description: Patient or doctor not found

components:
  schemas:

    PatientDTO:
      type: object

      properties:
        id: { type: integer, format: int64 }
        name: { type: string }
        email: { type: string }
        phone: { type: string }
        birthDate: { type: string }
        gender: { type: string }

    PatientRequest:
      type: object
      required: [name, email, phone]

      properties:
        name:
          type: string
          minLength: 2
          maxLength: 100

        email:

```

```

        type: string
        format: email

    phone:
        type: string
        pattern: '^[0-9+\\-\\s]{7,20}$'

    birthDate: { type: string }
    gender: { type: string }

DoctorDTO:
    type: object

    properties:
        id: { type: integer, format: int64 }
        name: { type: string }
        email: { type: string }
        phone: { type: string }
        specialty: { type: string }
        available: { type: boolean }

DoctorRequest:
    type: object
    required: [name, email, phone, specialty]

    properties:
        name: { type: string, minLength: 2, maxLength: 100 }
        email: { type: string, format: email }
        phone: { type: string }
        specialty: { type: string }
        available: { type: boolean, default: true }

AppointmentDTO:
    type: object

    properties:
        id: { type: integer, format: int64 }
        patientId: { type: integer, format: int64 }
        doctorId: { type: integer, format: int64 }
        date: { type: string }
        timeSlot: { type: string }
        reason: { type: string }
        status: { type: string }

AppointmentRequest:
    type: object
    required: [patientId, doctorId, date, timeSlot]

    properties:
        patientId: { type: integer, format: int64 }
        doctorId: { type: integer, format: int64 }
        date: { type: string }
        timeSlot: { type: string }
        reason: { type: string }

BookingRequest:
    type: object

```

```
required: [patientId, doctorId, date, timeSlot]
```

```
properties:
```

```
  patientId: { type: integer, format: int64 }  
  doctorId: { type: integer, format: int64 }  
  date: { type: string, description: 'YYYY-MM-DD' }  
  timeSlot: { type: string, description: 'HH:MM' }  
  reason: { type: string }
```

```
BookingResponse:
```

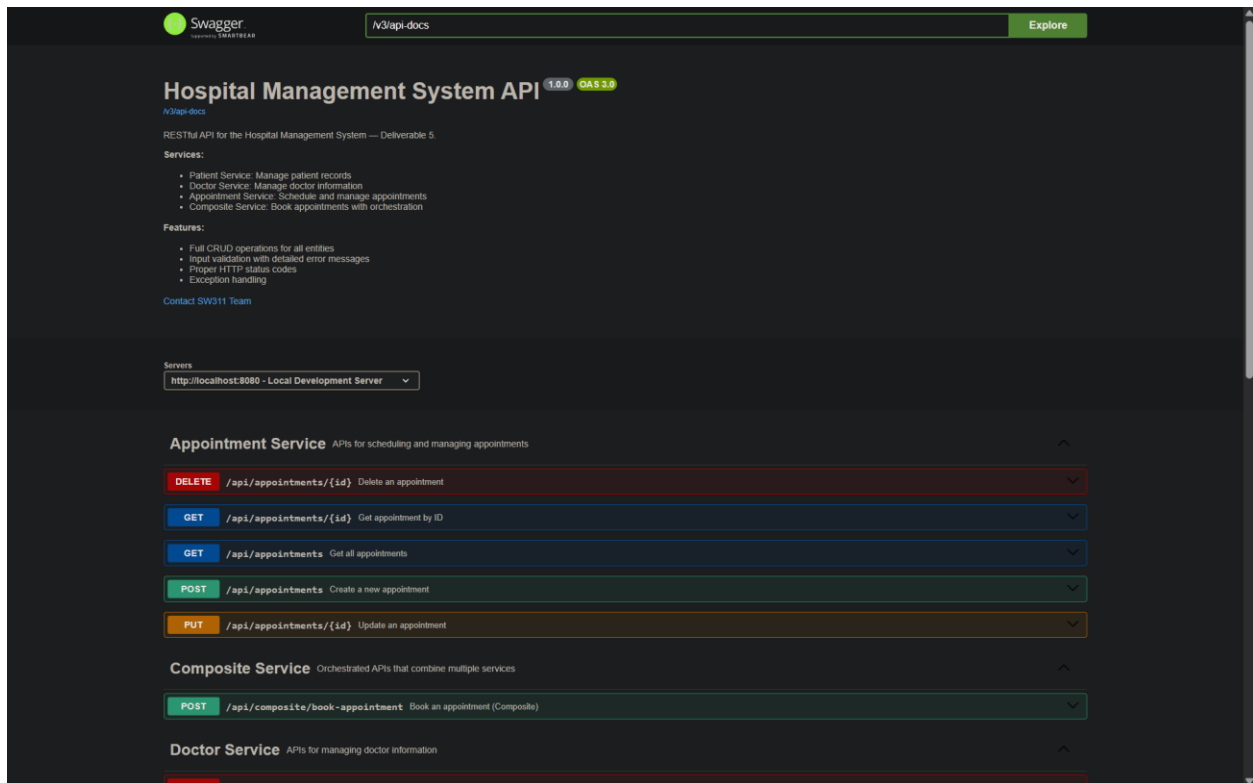
```
  type: object
```

```
properties:
```

```
  appointmentId: { type: integer, format: int64 }  
  patientId: { type: integer, format: int64 }  
  patientName: { type: string }  
  doctorId: { type: integer, format: int64 }  
  doctorName: { type: string }  
  doctorSpecialty: { type: string }  
  date: { type: string }  
  timeSlot: { type: string }  
  reason: { type: string }  
  status: { type: string }
```

API Documentation and Swagger UI

Swagger UI Implementation



Our Hospital Management System includes comprehensive API documentation through Swagger UI, accessible at:

- **Swagger UI:** <http://localhost:8080/swagger-ui/index.html>
- **OpenAPI JSON:** <http://localhost:8080/v3/api-docs>

The Swagger UI provides interactive documentation where users can:

- View all available API endpoints
- Explore request/response schemas
- Test endpoints directly from the browser using "**Try it out**"
- View example requests and responses

API Endpoints Overview

Service	Endpoint	Method	Description
Patient Service	/api/patients	GET	Retrieve all patients
	/api/patients/{id}	GET	Retrieve patient by ID

Service	Endpoint	Method	Description
Doctor Service	/api/patients	POST	Create new patient
	/api/patients/{id}	PUT	Update patient information
	/api/patients/{id}	DELETE	Delete patient
	/api/doctors	GET	Retrieve all doctors
	/api/doctors/{id}	GET	Retrieve doctor by ID
	/api/doctors	POST	Create new doctor
	/api/doctors/{id}	PUT	Update doctor information
Appointment Service	/api/doctors/{id}	DELETE	Delete doctor
	/api/appointments	GET	Retrieve all appointments
	/api/appointments/{id}	GET	Retrieve appointment by ID
	/api/appointments	POST	Create new appointment
	/api/appointments/{id}	PUT	Update appointment
Composite Service	/api/appointments/{id}	DELETE	Delete appointment
	/api/composite/book-appointment	POST	Book appointment (composite)

OpenAPI Specification

The complete API specification is documented in `openapi.yaml` with the following key components:

Info Section

- **Title:** Hospital Management System API
- **Version:** 1.0.0
- **Description:** RESTful API for HMS with Patient, Doctor, Appointment, and Composite services

Schemas Defined

- `PatientDTO` — Patient data structure
 - `DoctorDTO` — Doctor data structure
 - `AppointmentDTO` — Appointment data structure
 - `BookingRequest` — Composite booking request
 - `BookingResponse` — Composite booking response
-

Screenshots Placeholders

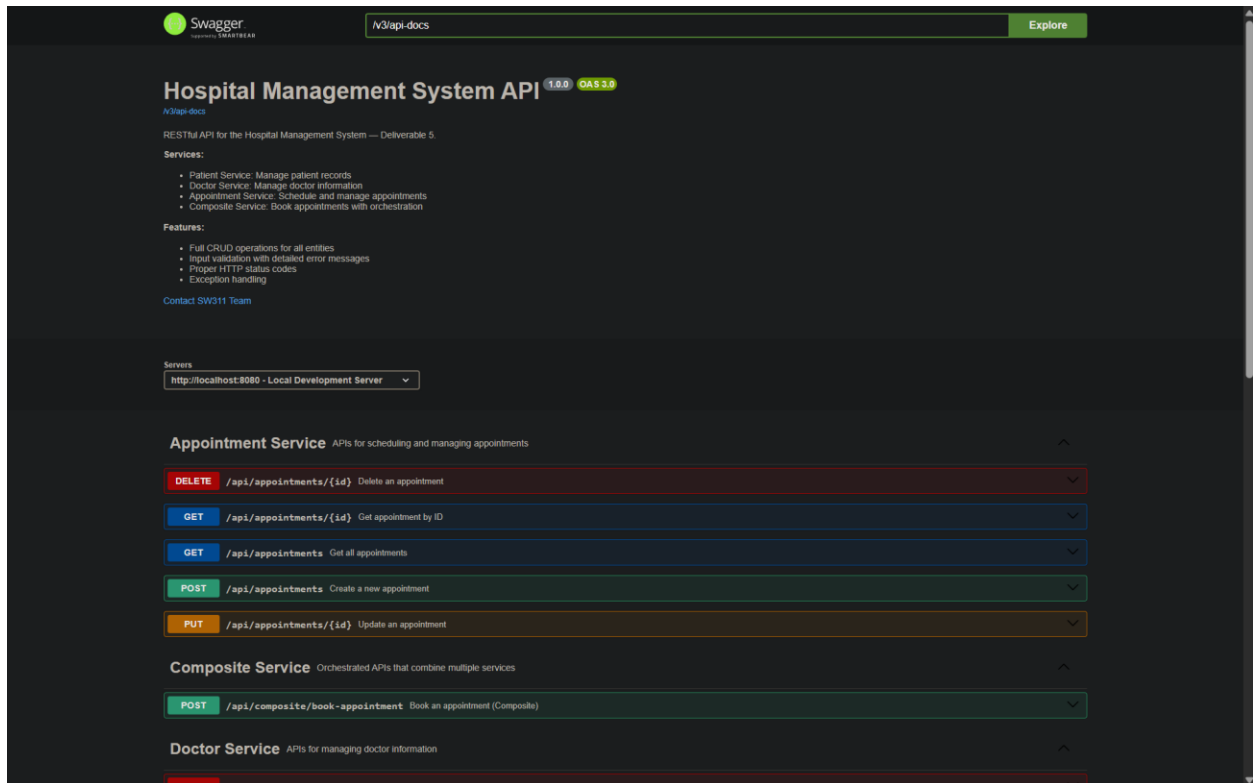


Figure 1: Swagger UI main page showing all HMS API endpoints

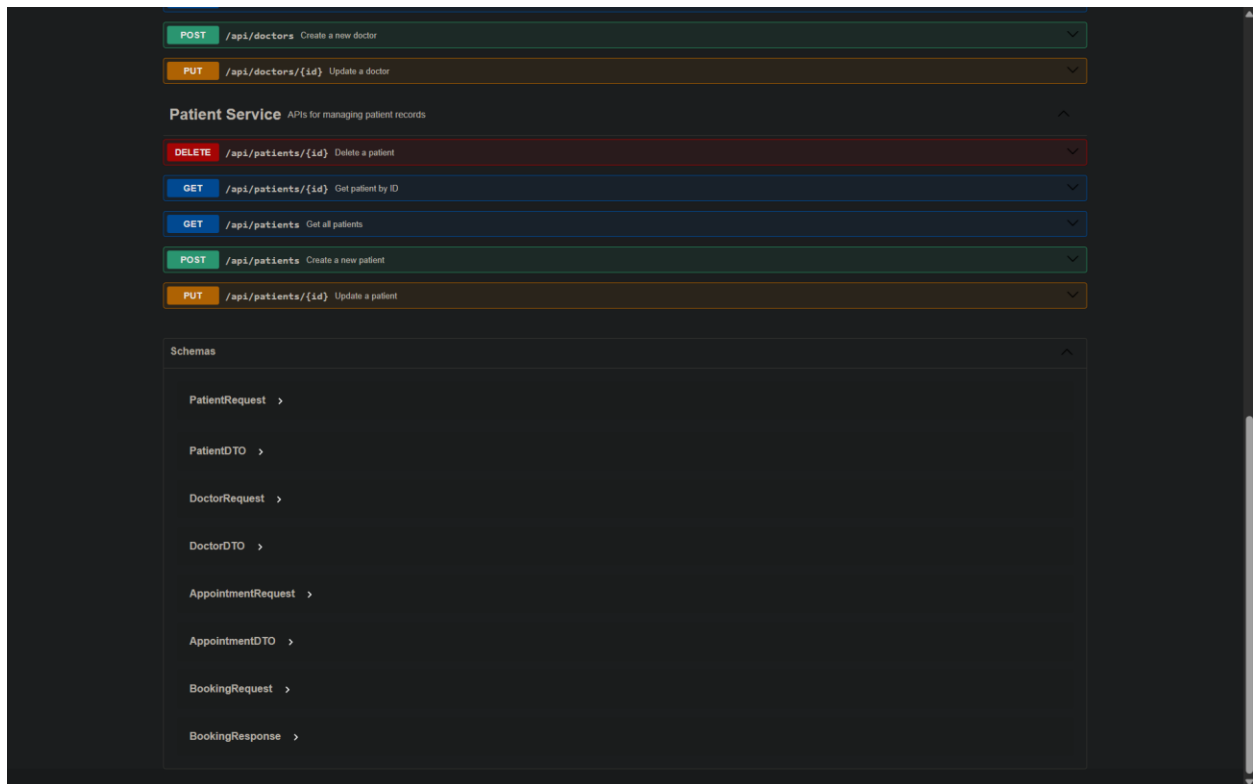


Figure 2: Patient Service endpoints expanded with schema details

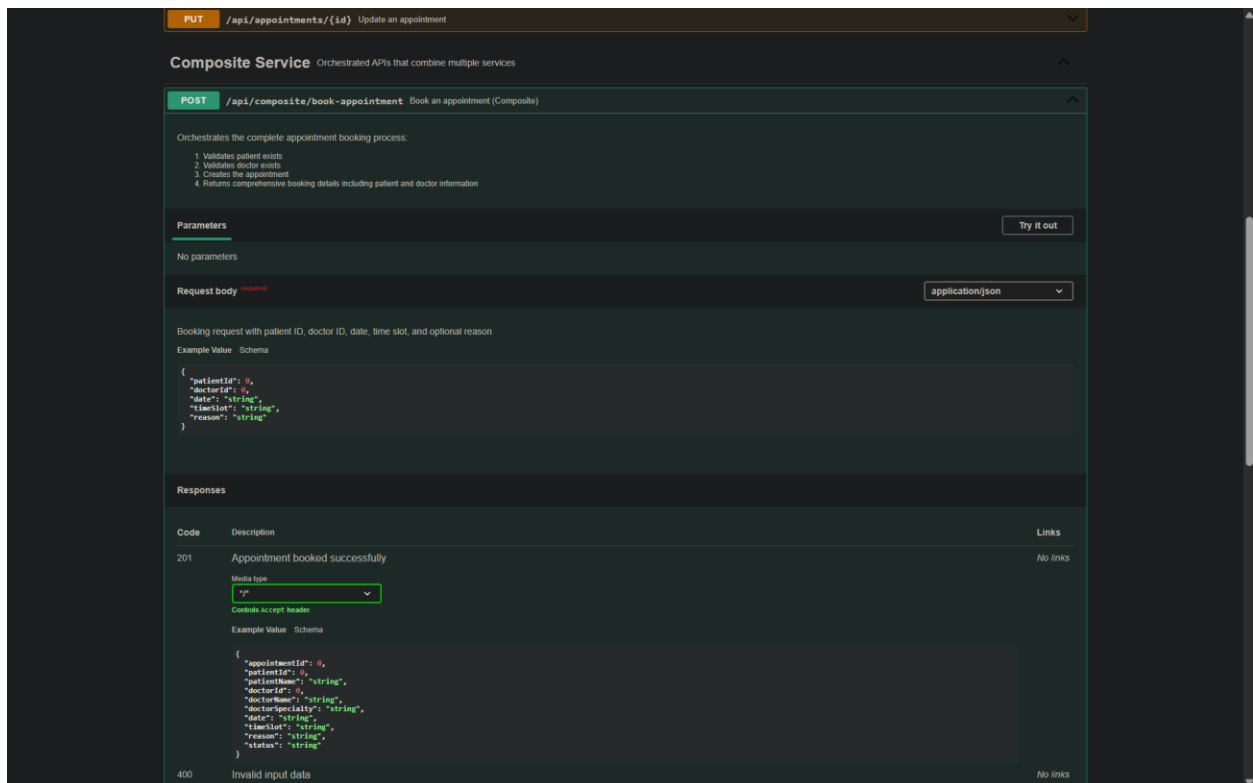


Figure 3: Composite book-appointment endpoint with request/response example

Documentation Features

The Swagger UI implementation includes:

- **Interactive Testing:** All endpoints can be tested directly from the browser
- **Schema Validation:** Automatic request/response validation
- **Error Documentation:** 400, 404, and 409 error responses documented
- **Example Data:** Pre-filled example requests for all endpoints
- **Response Examples:** Sample responses for successful operations

Web GUI Implementation

Our Hospital Management System includes a bonus web-based user interface that provides a user-friendly alternative to direct API calls.

Access and Features

- **HMS Portal:** <http://localhost:8080/>
- **Technology Stack:** HTML, CSS, JavaScript with Spring Boot backend integration

GUI Capabilities

The web interface provides complete HMS functionality.

Patient Management

- View all patients in a responsive table format
- Add new patients through web forms
- Edit existing patient information
- Delete patients with confirmation dialogs

Doctor Management

- Browse doctors with specialty information
- Add new doctors with availability settings
- Update doctor profiles
- Remove doctors from the system

Appointment System

- View appointment schedules
- Book new appointments with doctor selection
- Modify existing appointments
- Cancel appointments with reason tracking

Composite Booking

- Integrated appointment booking workflow
- Patient and doctor selection in a unified interface
- Real-time availability checking
- Confirmation with booking details

Technical Integration

The web GUI uses the same REST APIs documented in the Swagger UI section:

- All frontend operations call:
 - `/api/patients`
 - `/api/doctors`
 - `/api/appointments`
 - `/api/composite/book-appointment`
 - Real-time validation and error handling
 - Responsive design for desktop and mobile devices
-

GUI

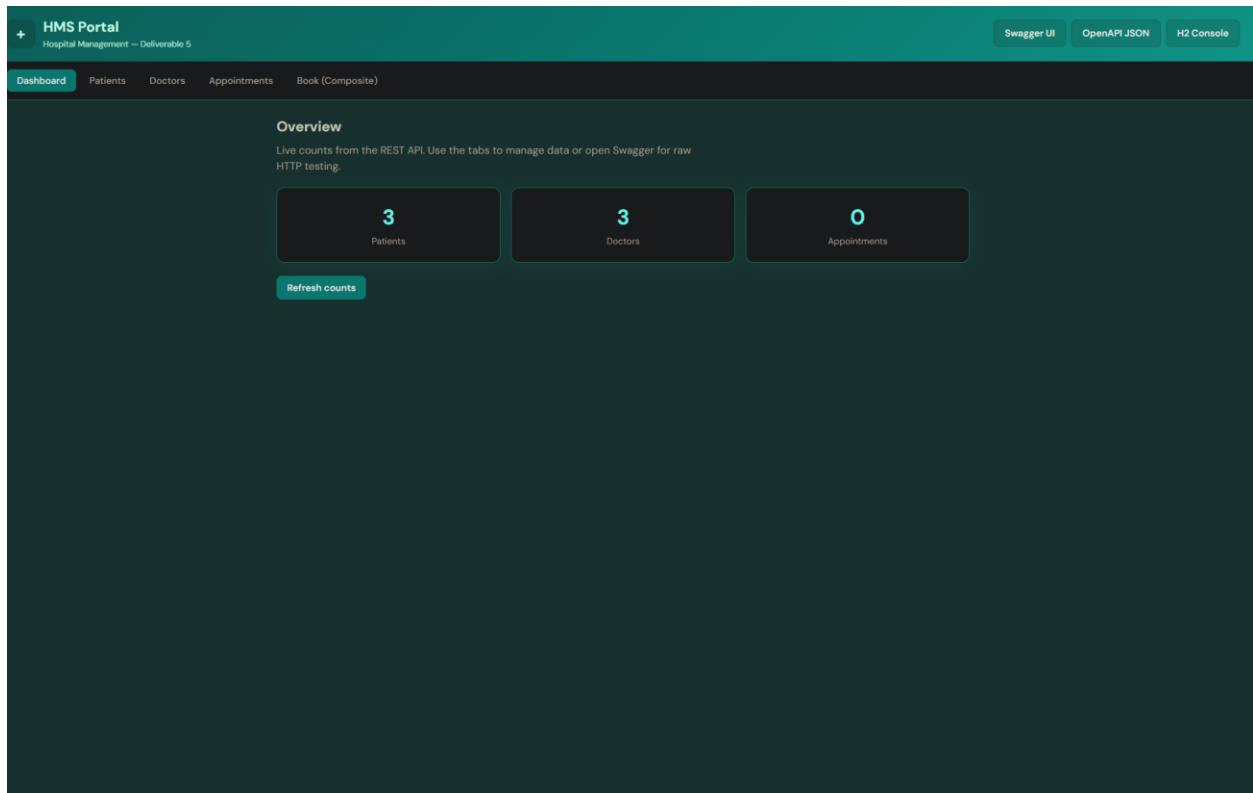


Figure 5: HMS Web GUI homepage with navigation menu

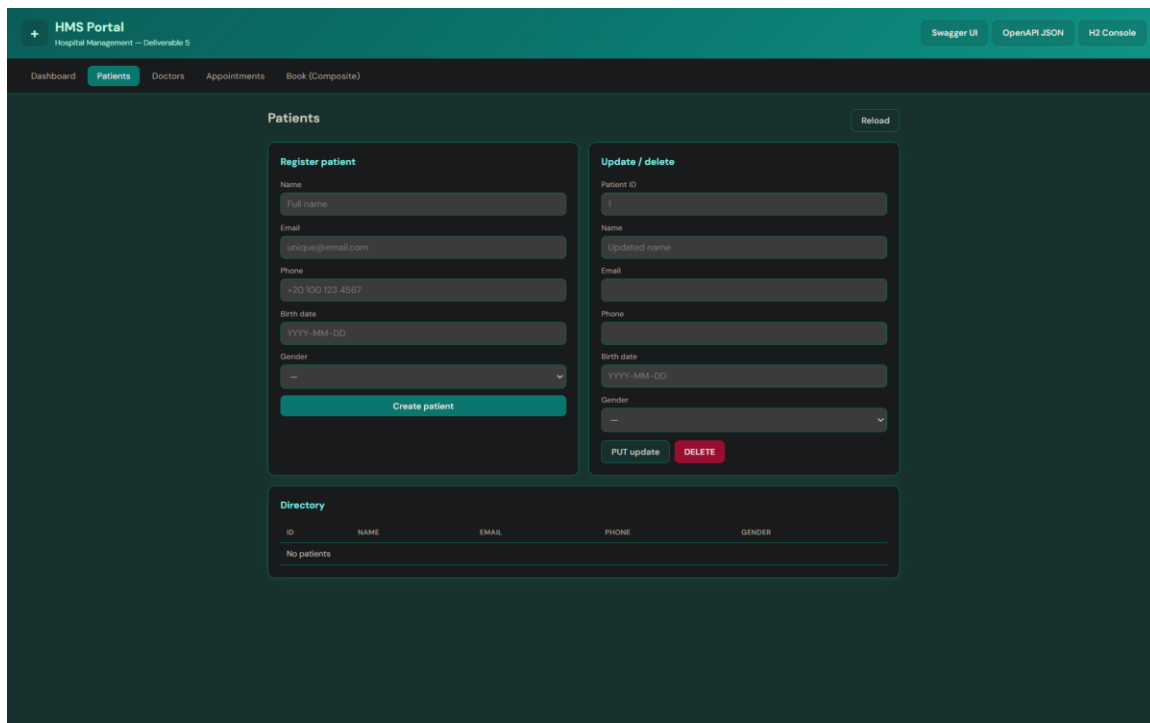


Figure 6: Patient management interface with CRUD operations

HMS Portal
Hospital Management — Deliverable 5

Swagger UI OpenAPI JSON H2 Console

Dashboard Patients Doctors Appointments **Book (Composite)**

Composite — Book appointment
Orchestrates patient check, doctor check, then creates the appointment in one call.

Patient ID

Doctor ID

Date
2026-06-01

Time slot
10:00

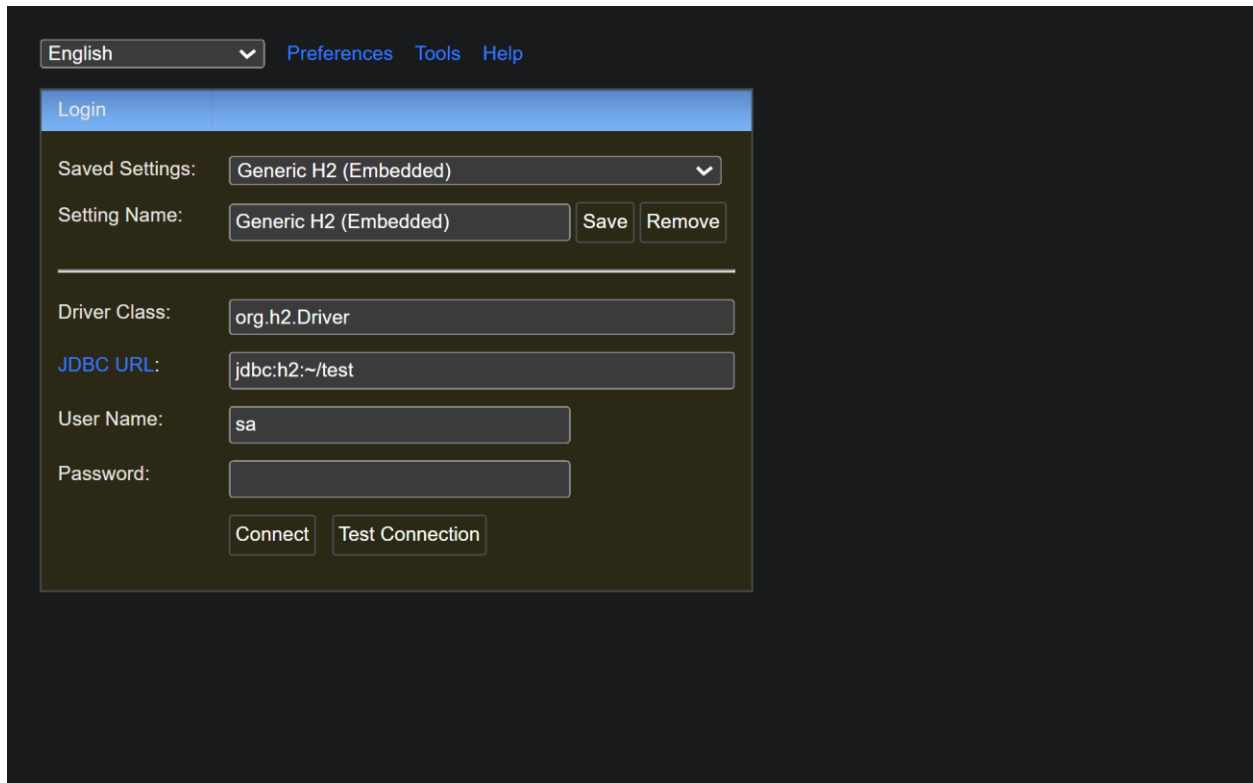
Reason
Routine checkup

POST /api/composite/book-appointment

Figure 7: Composite appointment booking interface

SECTION 2: Database Configuration

Database Configuration



The screenshot shows a 'Login' dialog box with a dark theme. At the top, there is a language dropdown set to 'English' and links for 'Preferences', 'Tools', and 'Help'. The 'Saved Settings' section shows 'Generic H2 (Embedded)' selected in a dropdown. Below this, the 'Setting Name' is also 'Generic H2 (Embedded)', with 'Save' and 'Remove' buttons. A horizontal line separates this from the configuration fields. The 'Driver Class' is 'org.h2.Driver'. The 'JDBC URL' is 'jdbc:h2:~/test'. The 'User Name' is 'sa'. The 'Password' field is empty. At the bottom are 'Connect' and 'Test Connection' buttons.

The HMS uses an H2 in-memory database for data persistence and management.

Database Setup

Configuration	Value
Database Type	H2 In-Memory Database
JDBC URL	jdbc:h2:mem:hmsdb
Username	sa
Password	(empty / blank)
Console Access	http://localhost:8080/h2-console

Database Schema

The system manages four main entities:

- **Patients** — Personal and contact information
- **Doctors** — Professional details and specialties

- **Appointments** — Booking information and scheduling
 - **Relationships** — Proper foreign key constraints between entities
-

Pre-loaded Data

On application startup, the system automatically populates sample data.

Sample Patients

1. **Ahmed Hassan** (`ahmed.hassan@example.com`)
2. **Sara Ali** (`sara.ali@example.com`)
3. **Mohamed Khaled** (`mohamed.khaled@example.com`)

Sample Doctors

1. **Dr. Fatima Zaki** — Cardiology
 2. **Dr. Omar Hamed** — Neurology
 3. **Dr. Layla Mostafa** — Pediatrics
-

Data Management Features

- **JPA/Hibernate Integration:** Automatic ORM mapping
 - **Entity Relationships:** Proper one-to-many and many-to-one mappings
 - **Data Validation:** Database-level constraints
 - **Transaction Management:** ACID compliance for data operations
-

Console Access

The H2 console provides direct database access for:

- Manual data inspection
 - SQL query testing
 - Database administration tasks
 - Debugging and verification
-

SECTION 3: Error Handling Details

Error Handling Strategy

The HMS implements comprehensive error handling with proper HTTP status codes and meaningful error messages.

HTTP Status Codes

400 Bad Request

Occurs for:

- Validation errors for missing required fields
- Invalid email formats
- Invalid phone number formats
- Malformed date/time inputs

Example: Missing patient name when creating a new patient.

404 Not Found

Occurs when:

- Patient ID does not exist
- Doctor ID does not exist
- Appointment ID does not exist

Example:

GET /api/patients/999 when patient 999 does not exist.

409 Conflict

Occurs for:

- Duplicate email addresses for patients
- Duplicate email addresses for doctors
- Appointment time slot conflicts

Example: Creating a patient with an email that already exists.

200 Success

Used for:

- Successful `GET` operations
- Successful `PUT` / `DELETE` operations
- Returning data in proper JSON format

201 Created

Used for:

- Successful `POST` operations
- Newly created resources

Includes:

- `Location` header with new resource URL

Error Response Format

All error responses follow a consistent JSON structure:

```
{
  "timestamp": "2026-05-12T10:30:00Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Email is required and must be valid",
  "path": "/api/patients"
}
```

Validation Examples

Patient Validation

Field	Validation Rules
Name	Required, 2–100 characters
Email	Required, valid email format, unique
Phone	Required, 7–20 characters, valid phone pattern

Field	Validation Rules
Birth Date	Required, valid date format
Gender	Required, valid enum value

Appointment Validation

Field	Validation Rules
Patient ID	Required, must exist
Doctor ID	Required, must exist
Date	Required, valid date format
Time Slot	Required, valid time format
Reason	Optional, maximum 500 characters

Error Prevention

The system includes both client-side and server-side validation.

Appendix C: Evidence Folder Contents

File / folder	Purpose
A0_Service_Scope_Master_Scenario.txt	Service scope and master scenario
A2_Patient_Service_Design.txt	Patient service full lifecycle design
A4_Composite_Service_Design.txt	Composite book-appointment design
B2_Doctor_Service_Design.txt	Doctor service full lifecycle design
M0_Appointment_Service_Design.txt	Appointment service design (coordination)
Y0_Service_Scenario.txt	Detailed composite API call sequence
B0_Jira_Screenshots/	Jira board screenshots + README_B0_JIRA.txt instructions

Postman Screenshots (project root folder):

- GET_patients.png
- GET_patient_by_id.png
- Error_404.png
- Error_400.png
- Error_409.png
- POST_doctor.png
- POST_appointment.png

- Composite_book_appointment.png
- PUT_patient.png
- PUT_appointment.png
- DELETE_patient.png
- DELETE_appointment.png